Primer Examen Regular
Análisis y Diseño de Sistemas II
Noviembre 2025

Federico Rodriguez

Ejercicio 1

Realizar ingeniería inversa y construir el modelo de diseño estático del siguiente código fuente

Ejercicio 2

Realizar ingeniería inversa y construir el modelo de diseño dinámico para el escenario de ejecución del método **main** del siguiente código fuente. Utilice un diagrama de secuencia.

```
public interface Notificable {
void recibirAviso(String msg);
}

public abstract class Persona {
private String nombre;

public Persona(String nombre) {
this.nombre = nombre;
}

public String getNombre() { return nombre; }
}

public class Empleado extends Persona {
private List<Pedido> historial = new ArrayList<>();

public Empleado(String nombre) { super(nombre); }

public void asignar(Pedido p) {
if (!historial.contains(p)) {
```

```
historial.add(p);
}
}

public List<Pedido> getHistorial() { return
Collections.unmodifiableList(historial); }
}

public class Cliente extends Persona implements Notificable {

private Notificador notificador;
private List<Pedido> pedidos = new ArrayList<>();

public Cliente(String nombre, Notificador not) {
super(nombre);
this.notificador = not;
}

public Pedido crearPedido(String id) {
Pedido p = new Pedido(id, this);
pedidos.add(p);
return p;
}

@Override
public void recibirAviso(String msg) {
System.out.println(getNombre() + " recibe aviso: " + msg);
}

public Notificador getNotificador() { return notificador; }

public List<Pedido> getPedidos() { return
Collections.unmodifiableList(pedidos); }
}

public class Notificador {

private Map<Cliente> suscriptores = new HashMap<>();

public void agregarSuscriptor(Cliente c) {
suscriptores.put(c);
}

public void enviar(String nombreCliente, String mensaje) {
Cliente c = suscriptores.get(nombreCliente);
```

```
if (c != null) c.recibirAviso(mensaje);
}
}

public class Pedido {

private String id;
private Cliente cliente;
private Empleado asignado;

public Pedido(String id, Cliente cliente) {
this.id = id;
this.cliente = cliente;
}

public void asignarEmpleado(Empleado e) {
this.asignado = e;
e.asignar(this);
}

public String getId() { return id; }
public Cliente getCliente() { return cliente; }
public Empleado getEmpleado() { return asignado; }
}

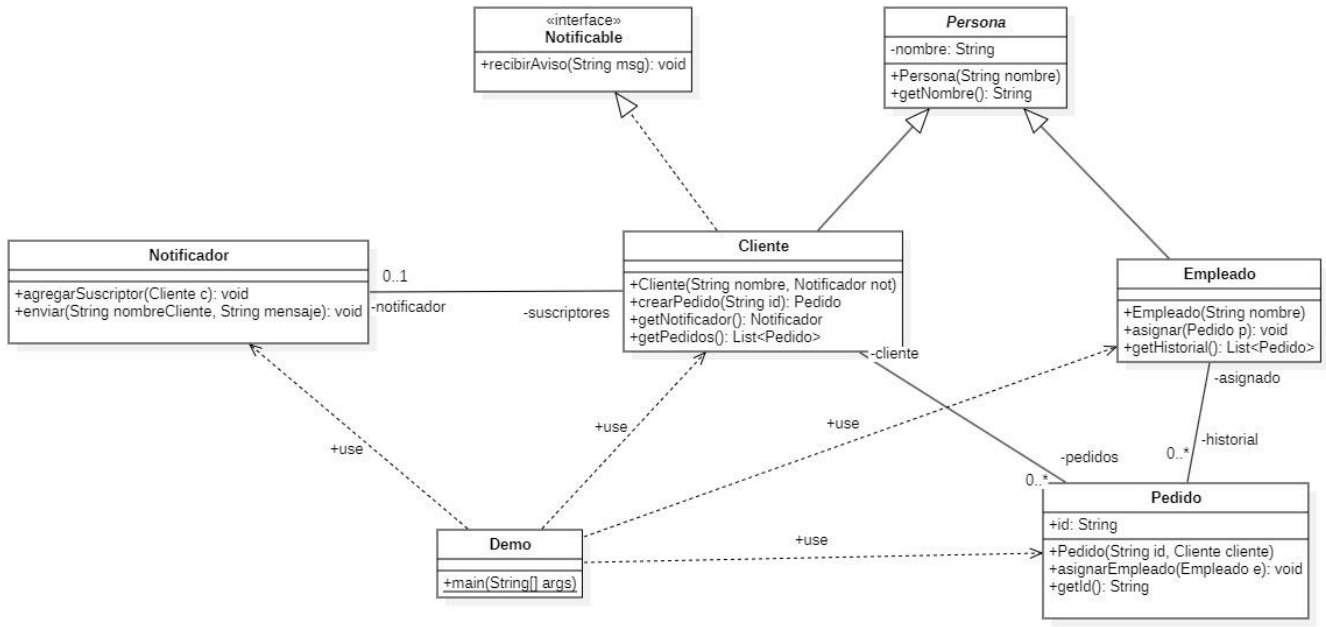
public class Demo {
public static void main(String[] args) {
Notificador not = new Notificador();

Cliente c1 = new Cliente("Ana", not);

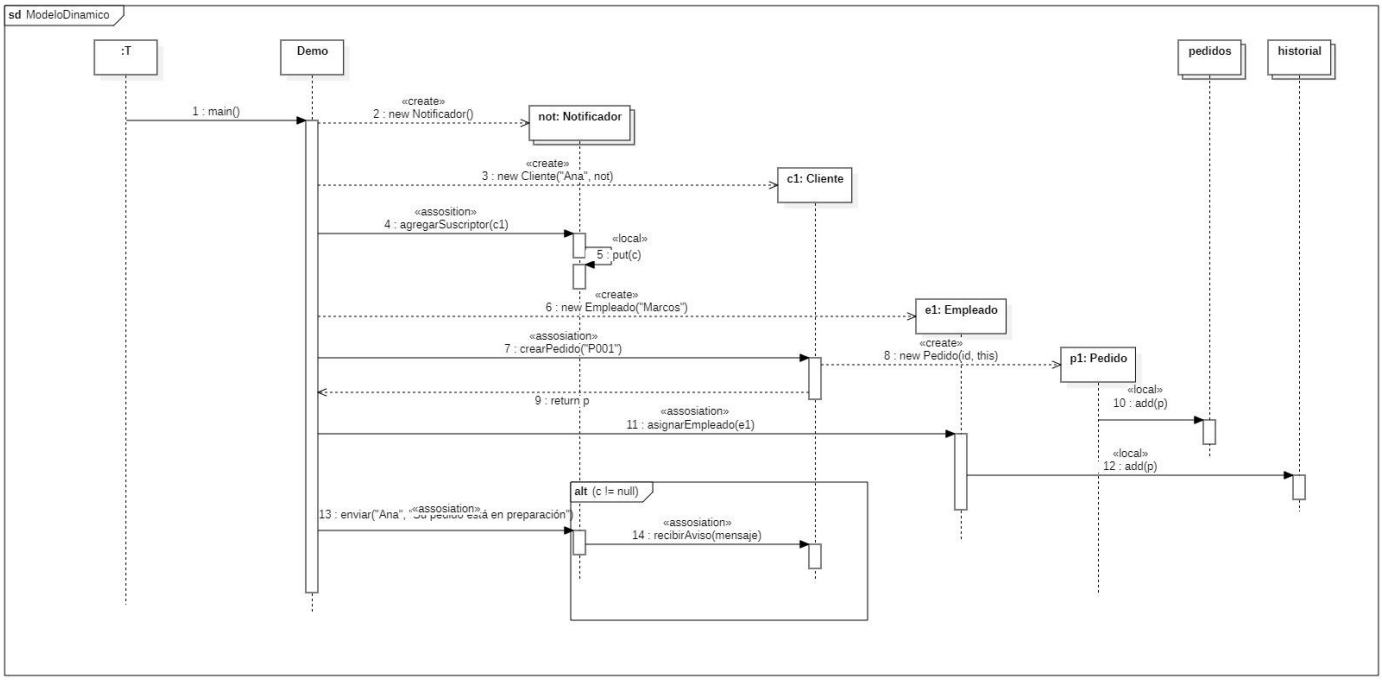
not.agregarSuscriptor(c1);

Empleado e1 = new Empleado("Marcos");
Pedido p1 = c1.crearPedido("P001");
p1.asignarEmpleado(e1);
not.enviar("Ana", "Su pedido está en preparación");
}
}
```

1)


«interface»
Notificable
+recibirAviso(String msg): void
Persona
-nombre: String
+Persona(String nombre)
+getNombre(): String
Notificador
+agregarSuscriptor(Cliente c): void
+enviar(String nombreCliente, String mensaje): void
0..1
-notificador
-suscriptores
Cliente
+Cliente(String nombre, Notificador not)
+crearPedido(String id): Pedido
+getNotificador(): Notificador
+getPedidos(): List<Pedido>
Empleado
+Empleado(String nombre)
+asignar(Pedido p): void
+getHistorial(): List<Pedido>
-cliente
-asignado
-historial
-pedidos
0..*
0..*
+use
+use
+use
+use
Demo
+main(String[] args)
Pedido
+id: String
+Pedido(String id, Cliente cliente)
+asignarEmpleado(Empleado e): void
+getId(): String

2)


sd ModeloDinamico
:T
Demo
pedidos
historial
1 : main()
«create»
2 : new Notificador()
not: Notificador
«create»
3 : new Cliente("Ana", not)
c1: Cliente
«assosition»
4 : agregarSuscriptor(c1)
«local»
5 : put(c)
«create»
6 : new Empleado("Marcos")
e1: Empleado
«assosiation»
7 : crearPedido("P001")
«create»
8 : new Pedido(id, this)
p1: Pedido
9 : return p
«local»
10 : add(p)
«assosiation»
11 : asignarEmpleado(e1)
«local»
12 : add(p)
alt (c != null)
13 : enviar("Ana", "Su pedido está en preparación")
«assosiation»
14 : recibirAviso(mensaje)